HACK HYDRA — TRACK 02A

HydraShield

Problem Statement, Solution Architecture & Competitive Positioning

Prepared for the Hack Hydra submission · August 2026

1. Executive Summary

Hack Hydra is HydraDB's nine-day open-source hackathon (August 12–20, 2026), with a $10,000 prize pool across three tracks. We are entering Track 02A: Repos, Dependencies + Code as Graphs, specifically the Supply Chain Blast Radius problem. The task, as HydraDB defines it, is to model the npm or PyPI dependency ecosystem as a graph in HydraDB and answer, in real time, which services are exposed when a package is compromised — a question the brief explicitly frames as a graph traversal problem that a vector database cannot answer.

Our proposed solution, HydraShield, builds a version-aware dependency graph in HydraDB and computes blast radius as a native graph traversal, with every answer accompanied by an explainable path from the compromised package to the exposed service. This document lays out the problem statement as HydraDB actually wrote it, what HydraDB's architecture actually is (verified against its documentation and website, not just marketing copy), our proposed graph model and system design, and — the part most plans skip — an honest answer to how we differentiate when every other team has the same AI assistants and can produce a similar-looking plan in minutes.

2. Hackathon Context

Hack Hydra is run directly by HydraDB to promote its newly open-sourced graph database. Key facts that shape strategy:

- **Format:** Online, teams of 1–4, build window August 12–20, 2026. Submission deadline August 20, 11:59 PM PT.
- **Prizes:** $5,000 Grand Champion, $3,000 Overall Runner-Up, $1,500 Third Place, $500 Best Use of HydraDB (judged separately, open to any eligible entry including finalists).
- **Advancement:** The top submission from each of the three tracks advances to a final round, where finalists are ranked holistically for the top three prizes.
- **Judging criteria:** technical execution, use of HydraDB and graph-native approaches, product completeness and usability, quality of results, and originality.
- **Submission requirements:** a completed Google Form, a public GitHub repository with an open-source license and no participant commits before August 12, and a demo video of 3 minutes or less structured as problem → project → demo → HydraDB usage.
- **Disqualifiers that matter to us:** “HydraDB not used meaningfully” is listed explicitly as a disqualifying condition, alongside missing repos, missing licenses, and late submission. This is not a minor formality — see Section 6.

3. The Problem Statement (Track 02A, as written by HydraDB)

HydraDB's own framing: “Supply-chain attacks via npm and PyPI are surging, and current developer tools fail to give real-time, deep context on malicious dependencies. This is fundamentally a graph traversal problem, not a semantic similarity problem.” Track 02 offers two directions; we are pursuing Direction A.

3.1 Direction A — Supply Chain Blast Radius (our chosen path)

When a package is compromised, the system must answer:

- Which internal services are transitively exposed?
- Which version introduced the vulnerability?
- Which apps resolved the bad version while it was live?
- Which packages share maintainers or infrastructure with the compromised one?
- Are there likely typosquat packages nearby?
- What is the complete blast radius?

HydraDB's brief cites the May 2026 TanStack compromise as the reference incident: 84 malicious package artifacts were published across 42 packages within six minutes of the CI pipeline being breached, and the worm went on to hit Mistral AI, UiPath, and over 160 other npm and PyPI packages — self-propagating and persisting in .claude/ and .vscode/ directories in a way that survived npm uninstall. The stated defender's problem is speed: “When a package is compromised at 09:00, which of your services are exposed by 09:06?” That is a transitive reverse-dependency closure over an ecosystem graph with tens of millions of versioned nodes.

> **How this track is actually graded**
>
> Ground truth is sourced independently from OSV and the GitHub Advisory Database — not from data we control.
>
> Recent advisories are held out. Teams are given an older snapshot of the ecosystem graph and must generalize to advisories that land after that snapshot.
>
> Scoring is on precision, recall, query latency, and cost — not on how polished the demo narration sounds.

This detail matters more than it looks. It means the evaluation is a blind test against unseen advisories, not a replay of whatever scenario we script for the demo video. A submission tuned only to look good on a rehearsed walkthrough can still score poorly here — see Section 6.2.

4. HydraDB Architecture — What It Actually Is

This section is verified against HydraDB's own documentation (docs.hydradb.com), architecture page (hydradb.com), and benchmark site (research.hydradb.com), not just the hackathon marketing copy. It's worth separating two things HydraDB is simultaneously marketed as: (1) a raw graph database engine, open-sourced for this hackathon at github.com/hydra-db/hydradb, and (2) a hosted “context substrate” product built on that engine, oriented around agent Memory and Knowledge primitives. Track 02A needs (1). Most of HydraDB's own docs are written for (2). We should build directly against the graph engine, not assume the Memory/Knowledge API is the intended surface for a dependency graph.

4.1 Storage engine

HydraDB is a graph database built on object storage, described on its own site as 'GraphDB built on object storage: 10x cheaper, ultra fast, and purpose-built for modern AI workloads.' The storage layer is tiered:

- **Hot tier:** in-memory cache for actively needed, frequently accessed graph data.
- **Warm tier:** NVMe SSD.
- **Cold tier:** object storage (S3-native), used for archival.

Context moves fluidly between tiers based on a retention score blending salience, recency, and reuse — not static rules. For a dependency ecosystem graph this matters directly: actively-queried packages (the ones in a live incident) stay hot, while the long tail of the npm/PyPI graph can sit cheaply in cold storage without being discarded.

4.2 Read/write architecture

The engine separates writes, indexing, and reads into distinct node roles, all operating on the object-storage layout under a namespace (namespace/data/wal, namespace/data/value, namespace/index, namespace/manifest.json):

- **Writer Node:** accepts Cypher writes and appends them to a write-ahead log (WAL) plus a value log, persisted to S3 with a disk cache.
- **Indexer Node:** reads WAL entries and builds the index using GraphBLAS — a sparse linear-algebra library for graph computation. Traversal and adjacency operations are implemented as matrix operations, which is the actual reason graph queries stay fast at scale, not an implementation detail we can ignore.
- **Reader Node:** serves Cypher queries against the index plus any pending WAL entries, giving strongly consistent reads.

Isolation is enforced at the namespace level: a "database" is a fully isolated workspace (no cross-database aggregation, ever), and "collections" are logical partitions within it — useful if we want to separate, say, staging ecosystem data from a production snapshot.

4.3 Query model — this is the part our implementation has to respect

HydraDB queries and writes use Cypher, the same declarative graph query language used by Neo4j: pattern-matching traversals (MATCH clauses, variable-length paths, relationship direction) run natively inside the engine. The documented query pipeline is multi-stage and fuses several retrieval modes in a single call: metadata filtering, semantic and keyword (BM25) retrieval with reranking, and graph traversal, plus optional connector plugins for external sources. For a dependency graph, the relevant piece is graph traversal via Cypher — the semantic/vector and 100+-source connector plugins are built for the document/knowledge-base use case and are not what Track 02A needs.

The practical implication: **reverse-dependency traversal, blast-radius closure, and path explanation should be expressed as Cypher queries executed inside HydraDB, not as a BFS/DFS algorithm we hand-write in Python against data we've pulled out of HydraDB.** The latter uses HydraDB as a key-value store with a graph-shaped label on it, and risks the explicit disqualifier "HydraDB not used meaningfully." We return to this in Section 6.1.

4.4 Temporal / versioned graph

HydraDB stores knowledge as a versioned, append-only graph: updates commit new timestamped edges instead of overwriting old ones, so no history is lost and every past state remains addressable. Valid-time metadata on edges

lets a query resolve when a fact became true, when it changed, and what holds now — described on HydraDB's own site as 'git-style temporal versioning.' This is a native engine capability, not something we need to simulate with our own snapshot table, and it is exactly the primitive our temporal replay feature (Section 5.4) should be built on.

4.5 Independently reported performance

- 90.79% overall on LongMemEval-S, reported as best-in-class against mem0-OSS (29.07%), Zep (71.2%), and full-context GPT-4 (60.2%), current as of the site's March 2026 benchmark update.
- Sub-200ms retrieval latency target.
- 1B+ documents ingested and roughly 1M retrievals/month reported in production usage.
- SOC 2 and ISO 27001 certified; self-hosting available for data-residency requirements.

These numbers describe the hosted memory/knowledge product's benchmark suite (LongMemEval, BEAM, FinanceBench) — they are not evidence of npm/PyPI-scale performance, and we should not cite them as if they were. What they do confirm is that the underlying engine (tiered storage, GraphBLAS indexing, Cypher traversal) is a real, benchmarked system and not a prototype.

5. Our Solution — HydraShield

HydraShield is a graph-native supply-chain intelligence system built on HydraDB. Its core capability is blast-radius computation for a compromised npm or PyPI package: given a compromised PackageVersion, traverse the dependency graph backward to find every exposed repository, service, and production environment, and return an explainable path for each result, not just a list.

5.1 Graph model — version-aware, not package-aware

The key modeling decision is to make PackageVersion, not Package, the central node. Vulnerabilities bind to version ranges, so a graph that only knows about "lodash" cannot distinguish an exposed service from a safe one on a patched version.

Node types	Edge types
Organization, Repository, Service, Environment	Repository USES Lockfile
Package, PackageVersion	Lockfile RESOLVES_TO PackageVersion
Maintainer, Advisory, CVE	PackageVersion DEPENDS_ON PackageVersion
Lockfile, Commit, Build, Deployment	PackageVersion VERSION_OF Package
	Package MAINTAINED_BY Maintainer
	PackageVersion HAS_CVE CVE
	Repository DEPLOYED_TO Environment
	Commit INTRODUCED PackageVersion · Build PRODUCED Lockfile · Service RUNS Repository

5.2 Pipeline

Data sources (GitHub repos, package.json / package-lock.json / pnpm-lock.yaml / poetry.lock / requirements.txt, OSV advisories, GitHub Advisory Database, maintainer metadata) feed a Python ingestion layer that parses manifests and normalizes versions, then writes nodes and edges into HydraDB via Cypher. Blast-radius, path-explanation, and (where used) temporal-replay queries are executed as Cypher inside HydraDB. A FastAPI layer exposes results to a React dashboard and to a natural-language query translator that converts a security question into Cypher parameters, lets HydraDB execute the traversal, and asks an LLM only to summarize the result — the LLM never invents the answer or does the graph reasoning itself.

5.3 Reference query

The brief's own example: “Which production services are exposed to lodash 4.17.19?” This is a reverse-dependency closure — starting at the CVE, walking backward through HAS_CVE, DEPENDS_ON (reversed), RESOLVES_TO, USES, and DEPLOYED_TO to reach Environment = Production — expressed as a single variable-length Cypher MATCH rather than an application-level graph walk. This is the concrete test of whether we are ‘using HydraDB meaningfully’: if this query can only be answered by pulling the whole subgraph into Python and walking it there, we have built a JSON store with a HydraDB label on it.

5.4 Explainability

Every blast-radius answer returns the traversal path, not just the conclusion: Payment API → auth-lib@2.1 → lodash@4.17.19 → CVE-2026-1234. This directly targets the judging criterion “use of HydraDB and graph-native approaches” — a traceable path is something a vector-similarity system structurally cannot produce, since it has no notion of a path at all.

5.5 Feature scope: MVP vs. stretch

The original plan (ingestion, graph builder, reverse-dependency index, blast radius, temporal replay, maintainer trust graph, risk scoring, typosquat detection, natural-language layer — eight modules) is directionally right but overscoped for nine days by a small team. We reprioritize:

- **MVP (must ship, deeply tested):** ingestion → version-aware graph in HydraDB → native Cypher blast-radius traversal → explainable paths → a held-out evaluation harness → a working dashboard.
- **Stretch, in priority order:** (1) temporal replay built on HydraDB's native versioned edges, (2) maintainer trust graph, (3) typosquat detection, (4) natural-language query layer.

Stretch features are only added once the MVP passes its own held-out test (Section 6.2). This ordering is explained in Section 6.3.

6. Standing Out — Everyone Has Claude and ChatGPT Too

This is the real question, and it deserves a direct answer rather than a list of generic “we're different because we're innovative” claims. The Track 02A brief hands every team the same vocabulary: blast radius, reverse dependency, maintainer overlap, typosquat, the TanStack incident, even the shape of the example query. Any team that pastes this brief into Claude or ChatGPT will independently converge on a very similar plan — a PackageVersion-centric graph, reverse DEPENDS_ON traversal, FastAPI plus React, temporal replay, a maintainer graph, typosquat scoring.

Expect several finalists to submit something structurally close to this document. The idea is not the moat. What happens after the plan is.

6.1 Use HydraDB as a graph engine, not as storage

This is the single biggest risk in the original draft, and it is a risk every AI-assisted team shares, because it is the path of least resistance: pull dependency data into HydraDB, then write the actual blast-radius algorithm (BFS/DFS) in Python against data fetched back out. That satisfies "we used HydraDB" in the loosest possible sense while doing none of the graph reasoning inside it. The rules list "HydraDB not used meaningfully" as an explicit disqualifier, and judges are told to score "use of HydraDB and graph-native approaches" directly — and per the guide, "be ready to say where it is used and what the project would lose without it." Our differentiator: traversal, blast radius, and (if built) temporal replay run as native Cypher inside HydraDB, using its versioned-edge model for time-aware queries instead of a hand-rolled snapshot table. We should be able to answer the "what would we lose without HydraDB" question concretely, in the README and the demo video, because judges are explicitly primed to ask it.

6.2 Verify against the real evaluation, not a rehearsed demo

The brief describes a blind test: ground truth from OSV/GHSA, recent advisories held out, teams given an older ecosystem snapshot, scored on precision, recall, latency, and cost. A team that builds primarily by prompting an AI assistant for scaffolding will often demo well on a scripted scenario while never actually running this kind of held-out test — the scripted path and the general path silently diverge. Building our own held-out harness (ingest a real historical snapshot, simulate an advisory landing after it, and report actual precision/recall/latency/cost numbers in the README and video) is verification work, not idea generation, and it is the most defensible differentiator available because it can't be faked by a better prompt.

6.3 Cut scope on purpose

The original roadmap crams eight modules into nine days. Its own conclusion warns that "the strongest submission is not the one with the largest dataset" — but the roadmap it proposes maximizes feature count, which tends to produce eight shallow, under-tested features rather than three that survive the held-out evaluation. Because sprawling roadmaps are exactly what you get when you ask an LLM to plan a hackathon project, most competitors will likely have a similarly overstuffed plan. A team that visibly ships less — blast radius, explainability, one well-executed temporal or maintainer feature, verified against real held-out advisories — will look more credible than one demoing seven half-built ones. The judges say this directly: "We care about working, thoughtful products, not just benchmark scores," and their final piece of advice is "stop adding features before the deadline, test what you already built."

6.4 Submission hygiene as a scoring lever

The guide states plainly that most disqualifications are a missing link, not a weak project: public repo, correct license, no pre-August-12 commits, a README that explicitly explains HydraDB usage, setup instructions that actually work, and a demo video under three minutes structured exactly as required (problem → project → demo → HydraDB). None of this is generated for us by an AI assistant unless we explicitly drive it — it is pure execution discipline, and it is free points that many technically stronger teams will drop.

6.5 Decide on Direction A vs. B deliberately

Track 02 has a second, less crowded direction: code graphs for IDE assistants, evaluated on SWE-bench against a similarity-only baseline. Direction A (blast radius) is more dramatic in a demo and matches the TanStack narrative, which is likely why most teams will default to it — increasing overlap with other entries. We are recommending A, but this should be an explicit go/no-go decision made in the first hours of the build, not a default.

7. Revised 9-Day Execution Plan

Day	Deliverable
1	HydraDB setup, namespace/schema design, Cypher familiarization
2	Ingestion pipeline (npm/PyPI lockfiles + OSV/GHSA sync) writing directly into HydraDB
3	Full graph construction on a real historical ecosystem snapshot
4	Blast radius as native Cypher traversal, with path explanation
5	Held-out evaluation harness: withhold recent advisories, measure precision / recall / latency / cost
6	Dashboard + explainable-path UI on real (not scripted) results
7	Stretch #1: temporal replay using HydraDB's native versioned edges — only if MVP passes its own eval
8	Stretch #2: maintainer trust graph — only if time remains; otherwise polish and widen the eval set
9	README (HydraDB usage explained explicitly), demo video, submission form, feature freeze

8. Submission Checklist

Directly from the Hack Hydra participant guide — check every line before submitting, since most disqualifications are a missing item, not a weak project:

- Submission form complete (project name, problem, what was built, tech stack, HydraDB usage, team contributions, repo + video links)
- GitHub repository is public, licensed, and contains no participant commits dated before August 12, 2026
- README clearly explains the problem, the graph model, and specifically how HydraDB is used and what the project would lose without it
- Setup instructions actually work end to end
- Demo video ≤ 3 minutes, in order: the problem, the project, the demo, HydraDB usage
- Held-out evaluation results (precision, recall, latency, cost) included in the README or video, not just a scripted demo
- Submitted before 11:59 PM PT, August 20, 2026